Universidad Carlos III de Madrid

Escuela Politécnica Superior

APUNTES • IA con inspiración biológica.

Aprendizaje evolutivo de la representación de los datos para modelos basados en distancias

Andrea López Salazar

Máster en Ciencia y Tecnología
Curso 2026–2027
6 de noviembre de 2026

1 Problema

Nos piden un programa P que, dado un conjunto de entrenamiento (X_t, y_t), aprenda con técnicas de computación evolutiva una transformación del los atributos: síntesis de atributos o reducción de dimensionalidad. Esa transformación se aplica después al conjunto de validación: $X'_v = v(X_v)$.

Si entrenamos un modelo arbitrario M con los atributos originales y medimos el error absoluto medio (MAE) en validación, obtenemos e. Si entrenamos el mismo m odelo M con los atributos transformados y obtenemos e', el método funciona cuando $e'<e$.

1.1 Planteamiento

- El modelo M es arbitrario. Por eso evaluamos siempre con varios modelos — regresión lineal, KNN y árbol de decisión — y no con uno solo.
- La transformación se aprende solo con el entrenamiento, siendo la validación lo único que usamos para medir.
- Buscamos un método generalizable que funcione igual en datasets distintos sin ajustes a mano.

1.2 Datos

Partimos de dos datasets de regresión: Diabetes (442 muestras, 10 atributos) y California Housing (20640 muestras, 8 atributos). Para comprobar que el método es generalizable, se usan de manera adicional 10 datasets de OpenML — la lista y sus tamaños los tenemos en una sección posterior.

1.3 Protocolo común

- Usamos una partición con 70 % de entrenamiento y 30 % de validación con semillas distintas para cada unas de las repeticiones.
- La métrica va a ser el MAE en el conjunto de la validación.
- Para poder comprobar datasets con escalas muy distintas, usamos el MAE relativo a los datos originales, y el MAE/MAE(crudo), promediado con la media geométrica. Si es menor que 1, significa que hemos mejorado.
- Contrastes estadísticos, incluido el test de Wilcoxon por pares, aplicado sobre el cociente de errores de cada pareja (dataset, partición).

2 Teoría

2.1 Modelos basados en distancias: dependencia de la representación

El KNN predice el valor de un punto nuevo promediando el valor de sus k vecinos más cercanos. La cercanía la defina la distancia, usualmente la euclídea:

$$d(x, x')^2 = \sum_{j=1}^{n} (x_j - x'_j)^2 \tag{1}$$

Esta fórmula trata a todos los atributos por igual, lo que supone un problema porque:

- Si un atributo se mide en miles, y otro en unidades, el primero dominará la distancia, incluso cuando no tiene por qué ser el más informativo.
- Los atributos de menor importancia añaden ruido a la distancia, por lo que muchos vecinos dejan de ser vecinos verdaderos: maldición de la dimensionalidad.
- Con atributos muy sesgados o con valores extremos, unos pocos puntos acaparan prácticamente todo el rango y el resto queda comprimido.

Un árbol de decisión no sufre nada de esto, porque solo compara los atributos con umbrales. Cualquier transformación monótona no cambiará el orden de los datos, y por ende, tampoco el árbol. LO usamos como control: si la transformación aprendida no cambia el error del árbol, es lo que esperamos, y por ende, confirma que la mejora en KNN viene de la geometría y no de otra cosa.

2.2 Transformación aprendida

Para cada atributo j aprendemos dos números, un exponente λ_j y un peso w_j.

2.2.1 Yeo-Johnson

Aquí tenemos en cuenta los exponentes Yeo-Johnson. Es una familia de transformaciones de potencia, que a diferencia de otras como Box-Cox, admite valores negativos y cero.

$$\psi(x;\lambda) = \begin{cases} \dfrac{(x+1)^\lambda - 1}{\lambda} & x \geq 0,\ \lambda \neq 0 \\ \log(x+1) & x \geq 0,\ \lambda = 0 \\ -\dfrac{(-x+1)^{2-\lambda} - 1}{2-\lambda} & x < 0,\ \lambda \neq 2 \\ -\log(-x+1) & x < 0,\ \lambda = 2 \end{cases} \tag{2}$$

Con $\lambda = 1$ la transformación es la identidad; con valores menores a 1, comprime los valores grandes — útil en presencia de colas largas a la derecha — y con valores mayores a 1 los expande.

2.2.2 Estandarización y peso

Después de transformar, centramos y escalamos con la media μ_j y la desviación σ_j del entrenamiento y multiplicamos por el peso.

$$z_j = w_j \cdot \frac{\psi(x_j;\lambda_j) - \mu_j}{\sigma_j} \tag{3}$$

Con esto, la distancia euclídea en el espacio transformado es una distancia ponderada en el espacio original:

$$d_\theta(x, x')^2 = \sum_j w_j (\tilde{x_j} - \tilde{x_j^\prime}) \tag{4}$$

Donde $\tilde{x}_j$ es el atributo transformado y estandarizado. El peso w_j^2 es la importancia del atributo j en la distancia. Si $w_j = 0$, el atributo desaparece, y si w_j es grande domina. En términos de aprendizaje de

métricas, es una **métrica diagonal**: matriz de Mahalanobis diagonal, con un paso previo de transformación no lineal por atributo.

2.2.3 Parámetros

El vector que buscamos es $\theta = (\lambda_1, ..., \lambda_n, w_1, ..., w_n)$ con dimensión $2n$ (16 en California y 20 en Diabetes). Lo limitamos a $\lambda_j \in [-2, 2]$ y $w_j \in [0, 3]$.

2.2.4 Punto de partida

Con $\theta_0 = (1, ..., 1)$, la transformación es exactamente la estandarización. Por eso la búsqueda arranca de la línea base "estándar" y cualquier mejora que encuentre es mejora real sobre ella. Esta propiedad la usamos siempre al diseñar los experimentos.

2.2.5 Limitación de λ

Sin límites, el estimador de máxima verosimilitud de Yeo-Johnson dio $\lambda \approx 13,6$ para la longitud de California. Con ese valor la columna se colapsaba prácticamente en un solo valor. Con el rango que se ha definido, eso no ocurre.

2.3 CMA-ES

CMA-ES (Covariance Matrix Adaptation Evolution Strategy) es una estrategia evolutiva para optimización continua sin gradiente. Es la técnica evolutiva que usaremos para buscar θ.

Mantiene una distribución normal multivariante sobre el espacio de búsqueda con media m, tamaño de paso σ y matriz de covarianza C. En cada generación:

1. Muestrea λ_{pop} candidatos: $x_k \sim m + \sigma\mathcal{N}(0, C)$.
2. Evalúa el fitness de cada uno y los ordena.
3. Selecciona los μ mejores y recalcula la media como una media ponderada entre ellos.
4. Adapta el valor de σ - con un camino de evolución que mide si los pasos van todos en la misma dirección o se cancelan - y C - con una actualización de rango uno y otra de rango μ - para que la distribución se alargue en las direcciones que funciona.

Esto nos conviene porque:

1. No necesita gradiente. Nuestro fitness incluye vecinos más cercanos y ajustes locales, que no son derivables.
2. Solo usa el orden de los candidatos, no su valor. Es invariante a transformaciones monótonas del fitness y bastante robusto al ruido.
3. Aprende la forma del problema - qué parámetros están correlacionados o necesitan pasos distintos - con pocos parámetros propios. El tamaño de la población por defecto es: $\lambda_{pop} = 4 + \lfloor 3 \ln D \rfloor$

2.3.1 Implementación

Usamos una implementación propia en la práctica, pero en los experimentos de esta extensión usamos la librería de referencia 'cma' (Hansen), con la que obtenemos resultados equivalentes. Trabajamos en el cubo normalizado $[0, 1]^D$ y reescalamos a los límites de cada parámetro.

2.3.2 Presupuesto

Cada evaluación del fitness es lo más caro — validación cruzada con KNN o con el ajuste local. Usamos 1000 evaluaciones en los experimentos de la extensión y 2020 en el notebook principal. En todas las comparacioens, todos los métodos tienen el mismo presupuesto.

2.4 El fitness. Cómo sabe la búsqueda que un θ es bueno

Para cada candidato θ medimos el error de un modelo sobre el entrenamiento usando validación cruzada de 5 pliegues. Pasos:

1. Subsample de hasta 2000 filas del entrenamiento - para que cada evaluación sea rápida.
2. Transformamos con θ, con la media y la desviación calculadas sobre esas filas.
3. Para cada pliegue ajustamos el modelo y medimos el MAE en el pliegue de prueba.
4. Dividimos entre $s = \overline{|y - \text{mediana}(y)|}$, el error de un predictor trivial que siempre dice la mediana:

$$f(\theta) = \frac{1}{5} \sum_{\text{pliegues}} \frac{MAE_{\text{pliegue}}(\theta)}{s} \tag{5}$$

El fitness es una fracción del error trivial: 0.5 significa que el modelo comeye la mitad de error que un predictor sin información. Esa normalización permite comparar datasets con escalas diferentes. En el notebook principal el fitness promedia regresión lineal y KNN. En la extensión usa el modelo para el que optimizamos: KNN o la regla local lineal.

2.4.1 Riesgo de sobreajuste al fitness

La búsqueda optimiza uan estimación del error, no el error real. Con pocos datos (Diabetes tiene 309 muestras de entrenamiento) la búsqueda aprende el ruido de esos pliegues. Veremos que el fitness mejora y la validación no. Es el problema central que tenemos con el dataset de Diabetes.

2.5 La regla de predicción. De KNN a regresión lineal local.

KNN es una regresión local constante. La predicción es la media de los k vecinos. Funciona bien si la función es plana en el entorno, pero cuando hay pendiente comete un sesgo, sobre todo en los bordes de los datos.

La regresión lineal local ponderada ajusta, para cada punto de consulta q, un hiperplano a sus vecinos, y evalúa en q. Para los k vecinos z_j con valores y_i:

$$min_{\beta_0,\beta} \sum_{i=1}^{k} w_i(y_i - \beta_0 - \beta^T(z_i - q)) + r\overline{w}||\beta|| \tag{6}$$

Y la predicción es $\hat{y}(q) = \beta_0$. Como centramos en q, el término independiente es directamente la predicción.

1. Pesos de vecindad tricúbicos: $w_i = (1 - u_i)$ con $u_i = \frac{d_i}{d_{max}}$ de modo que lso vecinos lejanos pesan menos y el último pesa casi cero.
2. Ridge r: estabiliza el ajuste cuando hay pocos vecinos o atributos muy correlacionados. No penalizamos el término independiente.
3. Parámetros de la regla: k y r. En el método final, los aprende CMA-ES junto a θ.

Es la idea clásica de la *locally weighted regression* Atkenson, Moore y Schaal, 1997; y de la regresión polinómica local de la estadística no paramétrica. Lo que añadimos es optimizar la métrica de la distancia directamente para esta regla, en lugar de reutilizar la que era buena para KNN.

2.6 Qué hay en la literatura y dónde nos situamos

Aprender métricas para el KNN ya es un campo establecido. LMNN y NCA se plantean para la clarificación. Para la regresión, Weinberger y Tesauro aprenden la métrica minimizando el error de regresión por kernel con gradiente, y existe una adaptación de LMNN a regresión con algoritmo evolutivo. También se han publicado artículos donde se ponderan atributos con CMA-ES. Un ancho de banda por variable elegido por validación cruzada es, en la práctica, una métrica diagonal optimizada para el estimador de regresión no paramétrica y elimina de forma automática las variables irrelevantes. Cambiar la regla de predicción manteniendo una métrica fija lo hace DNNR, que usa información diferencial entre los vecinos.

Lo que no encontramos publicado es una combinación concreta de:

- Exponente de Yeo-Johnson y peso por atributo
- Optimización con CMA-ES

- Directamente contra la regla local lineal
- Evaluación de forma sistemática frente a controles y modelos de referencia.

3 Código del método

1. Primero se realiza la configuración y se establece el estilo de las gráficas.
2. Carga de los datos, se descargan también 10 datasets de OpenML. Nos quedamos solo con atributos numéricos y si un dataset tiene más de 10.000 filas, usamos submuestra fija de 10.000 para que el coste sea comparable.
3. `ajustar(xt, theta)` recibe el entrenamiento y un vector $\theta = (\lambda, w)$ y devuelve una función 'v' que transforma cualquier conjunto de datos. Lo importante es que la media y la desviación se calculan solo con el entrenamiento y se reutilizan para la validación.
4. Para las reglas de predicción tenemos KNN y regresión lineal local. Las dos funciones están escritas con NumPy por bloques para que sean rápidas, ya que el fitness las llama miles de veces. `knn_rapido` da el mismo resultado que el `KNeighborsRegressor` (salvo en empates entre distancias iguales) y el `llr` es la regresión lineal local de la que se ha hablado en la teoría.
5. El fitness: cada función `hacer_fitness_*` devuelve una función `f(theta)` que se puede pasar a cualquier optimizador. Los pliegues de validación cruzada se fijan una vez (con la semilla), de modo que todos los candidatos se evalúan sobre los mismos pliegues y la comparación entre ellos es justa.
6. La búsqueda con CMA-ES se realiza con la función `buscar_cma` que envuelve la librería de `cma`. Trabajamos como hemos dicho antes en el cubo $[0, 1]^D$y reescalamos a los límites reales de cada parámetro, guardando el mejor fitness hallado tras cada evaluación para poder dibujar la curva de convergencia. La búsqueda arranca en θ_0 (la estandarización), con un peso inicial de $\sigma_0 = 0.2$ (un 20 % del rango).
7. Elegimos la k y la regularización sin usar la validación. En las configuraciones que no aprenden esos parámetros con CMA-ES (las líneas base), los elegimos por la validación cruzada dentro del entrenamiento, de modo que la comparación sea justa.

4 Demostración

Ejecutamos el método completo sobre una partición de cada dataset de la práctica